

Batch/Lot Tracking on the Operations Grid

How Batch, Batch Qty and Expiry Date work on the Inventory Operations grid across Indent, Inventory, Purchase and Sales — one shared screen, four business flows, and only one place a brand-new batch is ever minted.

Module harleys_customization (view + JS/SCSS only) **Status** Working; \$7 governance requirement not yet built

Scope stock.picking / stock.move / stock.move.line / stock.lot

CONTENTS

1. One screen, four business flows
2. The three Batch columns
3. Where batches can be created vs only selected
4. Universality — a batch created once is visible everywhere
5. Line-item batch add/delete — currently unrestricted
6. Not yet built: "only a higher official can change batch details"
7. Module reference
8. Known trade-offs / open items

1. One screen, four business flows

Indent, Inventory, Purchase and Sales don't have four separate Operations screens — they all render through the *same* Odoo form, `stock.view_picking_form`, extended once by

`harleys_customization/views/stock_picking_views.xml` (`view_picking_form_custom`). That inherited view isn't scoped to any specific action or menu, so every picking gets the same Operations-tab extensions regardless of how it was created. The four flows only diverge at the *field* level, via

`picking_type_code` on `stock.picking` (`picking_code` — the same thing — related onto `stock.move`):

BUSINESS FLOW	PICKING_TYPE_CODE	ENTRY POINT
Indent Request → Internal Transfer	<code>internal</code>	Indent Request form → "Internal Transfer"
Inventory → Internal Transfer	<code>internal</code>	Inventory app → Internal Transfers
Purchase → Receipt (GRN)	<code>incoming</code>	Purchase Order → Receipt
Sales → Delivery	<code>outgoing</code>	Sales Order → Delivery

INDENT AND INVENTORY ARE LITERALLY INDISTINGUISHABLE BY `PICKING_TYPE_CODE`

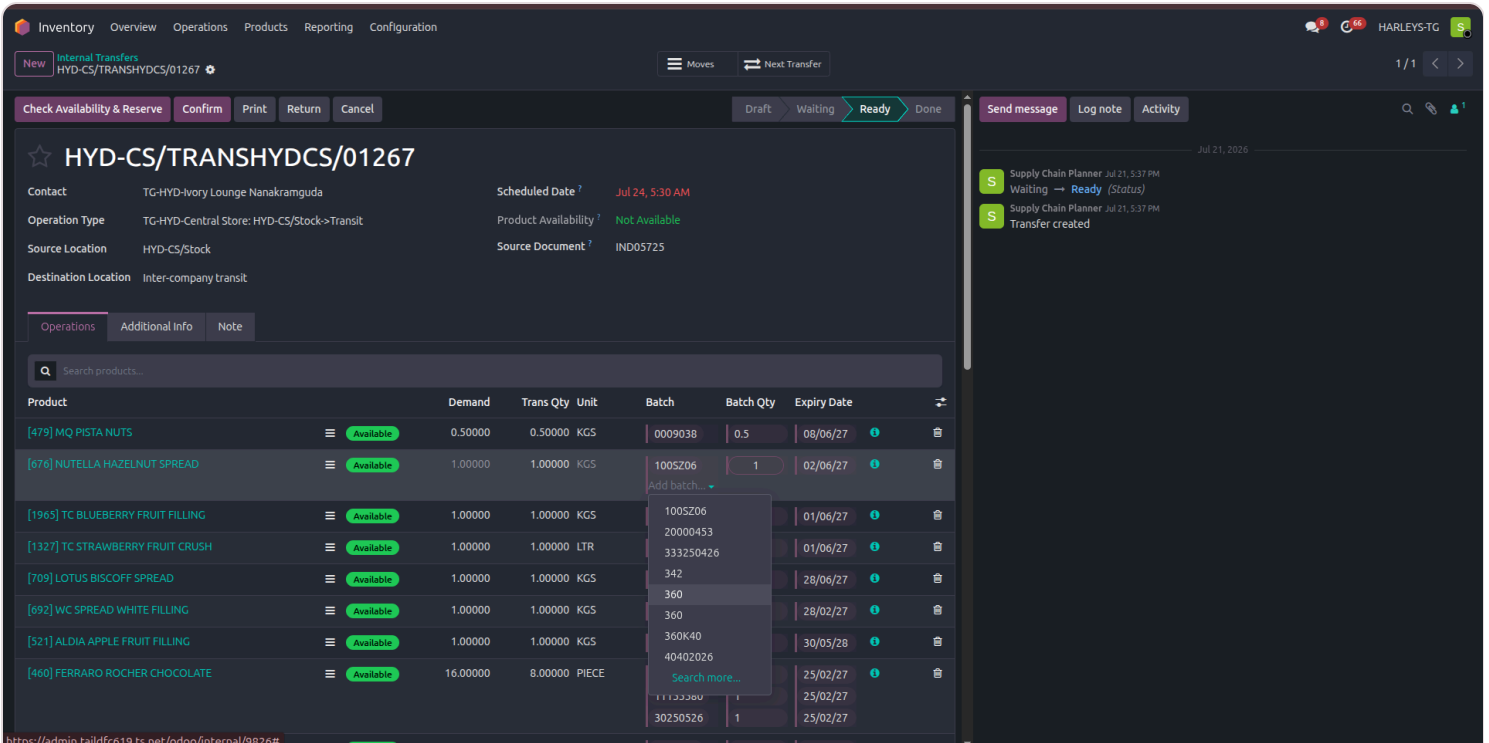
Both are `internal`. The only way to tell them apart in the data is whether a `stock.picking` is linked from an `indent.request` record — the Operations grid itself treats them identically, as the two screenshots below show for the exact same underlying record.

1 Indent Request → Internal Transfer

The screenshot shows the 'Indent Request' form for the record 'HYD-CS/TRANSHYDCS/01267'. The form is in the 'Ready' state. The 'Operations' tab is active, displaying a table of products. The 'Add batch...' dropdown menu is open, showing a list of existing batches (100SZ06, 20000453, 333250426, 342, 360, 360K40, 40402026, 11133380, 30250526) and a 'Search more...' option. The status bar at the top indicates 'Ready'.

`indent_internal_transfers_receipt.png` — `HYD-CS/TRANSHYDCS/01267`. The "Add batch..." dropdown lists only existing batches (100SZ06, 20000453, 333250426, 342, 360, ...) plus "Search more..." — no create option, because this is an `internal` picking.

2 The same record, opened from Inventory → Internal Transfers



inventory_internal_transfers.png — same picking record as above, reached through a completely different menu. The Operations grid and its Batch dropdown behave identically either way, because both are the same `internal` picking type underneath.

2. The three Batch columns

COLUMN	FIELD	WIDGET	EDITABLE?
Batch	<code>lot_ids</code>	<code>stock_move_line_batch</code>	Add/select/delete
Batch Qty	<code>move_line_ids</code> (<code>quantity</code>)	<code>stock_move_line_row_field</code> , <code>options="{ 'row_field': 'quantity', 'editable': True }"</code>	Yes, numeric input
Expiry Date	<code>move_line_ids</code> (<code>expiration_date</code>)	<code>stock_move_line_row_field</code> , <code>options="{ 'row_field': 'expiration_date', 'date_format': 'dd/MM/yy' }"</code>	Read-only, <code>dd/MM/yy</code>

All three are `column_invisible` unless `picking_type_code` is one of `internal` / `incoming` / `outgoing` — specific to these four flows and absent from any other picking type Harleys might use.

All three read from one shared row list — `getMoveLineRows()` in `static/src/js/stock_move_line_rows.js` — built primarily off `lot_ids.records`, with a fallback for Receipt lines whose quantity is still 0 (Harleys forces new GRN lines to `quantity = 0`, which would otherwise make core's own `_compute_lot_ids` hide the row before the user can type a quantity). Because all three columns share this one function, adding or deleting a batch in one column is reflected in the other two with no separate bookkeeping.

3. Where batches can be created vs only selected

This is the key difference between the four flows:

- **Purchase GRN (`incoming`) only** — clicking "Add" opens a dedicated popover (`StockMoveLineBatchQuickCreate`): live search against existing `stock.lot` records for the current product, and the ability to type a brand-new batch number. If the product tracks expiry, an expiry date is mandatory, entered via Odoo's own calendar widget locked to `dd/MM/yy` and unable to be backdated. **This is the only place in the system a new batch/lot record is minted through this grid.**
- **Indent/Inventory internal transfer and Sales delivery (`internal` , `outgoing`)** — clicking "Add" opens a plain inline autocomplete (`Many2XAutocomplete`), domain-filtered to the line's own product (`domain=" [('product_id', '=', product_id)]"`). Select-only in practice: there's no dedicated "new batch" UI here the way the Receipt popover has one. (The underlying widget still technically supports create-by-typing if `use_create_lots` is on for that picking type and the view's `options="{ 'create': [...] }"` allows it — that's bare Odoo default behavior inherited from `Many2XAutocomplete` , not a designed create flow, and isn't how these three flows are meant to be used.)

This matches how the business actually works: batches physically originate at goods receipt, so GRN is the only point that should mint new ones — the other three flows just draw down against what already exists.

1 Receipt — an existing batch, with a "+ Create" option alongside it

The screenshot displays a software interface for managing purchase orders. The top navigation bar includes tabs for Purchase, Orders, Products, Reporting, and Configuration. Below this, a header section contains buttons for Confirm, Quality Checks, Print, Return, Create e-Waybill / Challan, and Cancel. The main content area shows a receipt for HYD-CS/IN/00526, with details such as Receive From (YADIAH MILK AGENCY), Operation Type (TG-HYD-Central Store: Receipts), Destination Location (HYD-CS/Stock), Scheduled Date (Jul 25, 6:59 PM), Deadline (Jul 25, 6:58 PM), and Source Document (P01385). A table lists products, with 'AMUL CHEESE BLOCK' highlighted. A popover menu is open for the 'AMUL CHEESE BLOCK' row, showing existing batch 3214 (dated 31/07/26) and a '+ Create' button. The right sidebar shows a timeline of events, including updates and transfers.

purchase_orders_create_new_batch.png — `HYD-CS/IN/00526`, product **AMUL CHEESE BLOCK**. The popover lists existing batch 3214 (dated 31/07/26) *and* a "+ Create" option — the option that doesn't exist on the other three flows.

2 Receipt — expiry date is mandatory, no backdating

Purchase Orders / P01385
HYD-CS/IN/00526

Moves Quality Checks

1/1

Confirm Quality Checks Print Return Create e-Waybill / Challan Cancel

Draft Ready Done

Send message Log note Activity

HYD-CS/IN/00526

Receive From: YADAAIAH MILK AGENCY
Operation Type: TG-HYD-Central Store: Receipts
Destination Location: HYD-CS/Stock

Scheduled Date: Jul 25, 6:59 PM
Deadline: Jul 25, 6:58 PM
Source Document: P01385

Operations Additional Info Note

Search products...

Product	Demand	Trans Qty	Unit	Batch	Batch Qty	Expiry Date
[1712] AMUL CHEESE BLOCK	52.00000	52.00000	KGS	1900	10	31/07/26
Add a Product						

3214

DD/MM/YY

< > July 2026

S	M	T	W	T	F	S
27	28	29	30	1	2	3
28	5	6	7	8	9	10
29	12	13	14	15	16	17
30	19	20	21	22	23	24
31	26	27	28	29	30	31
32	2	3	4	5	6	7

WH Lead-HYD Jul 25, 7:02 PM
Updated: HYD-CS/IN/00526. — Logged by KOMATIREDDY VINEETH REDDY.

WH Lead-HYD Jul 25, 7:01 PM
Updated: HYD-CS/IN/00526. — Logged by KOMATIREDDY VINEETH REDDY.

WH Lead-HYD Jul 25, 7:00 PM
Updated: HYD-CS/IN/00526. — Logged by KOMATIREDDY VINEETH REDDY.

WH Lead-HYD Jul 25, 7:00 PM
Updated: HYD-CS/IN/00526. — Logged by KOMATIREDDY VINEETH REDDY.

OdooBot Jul 25, 6:59 PM
This transfer has been created from: P01385 — Logged by KOMATIREDDY VINEETH REDDY.

OdooBot Jul 25, 6:59 PM
Created: HYD-CS/IN/00526. — Logged by KOMATIREDDY VINEETH REDDY.

OdooBot Jul 25, 6:59 PM
Transfer created — Logged by KOMATIREDDY VINEETH REDDY.

purchase_orders_create_new_batch_select_exp_date.png — same Receipt. The calendar widget is locked to **DD/MM/YY** and every date before today (the 27th) is greyed out and unselectable, so a new batch's expiry can never be backdated.

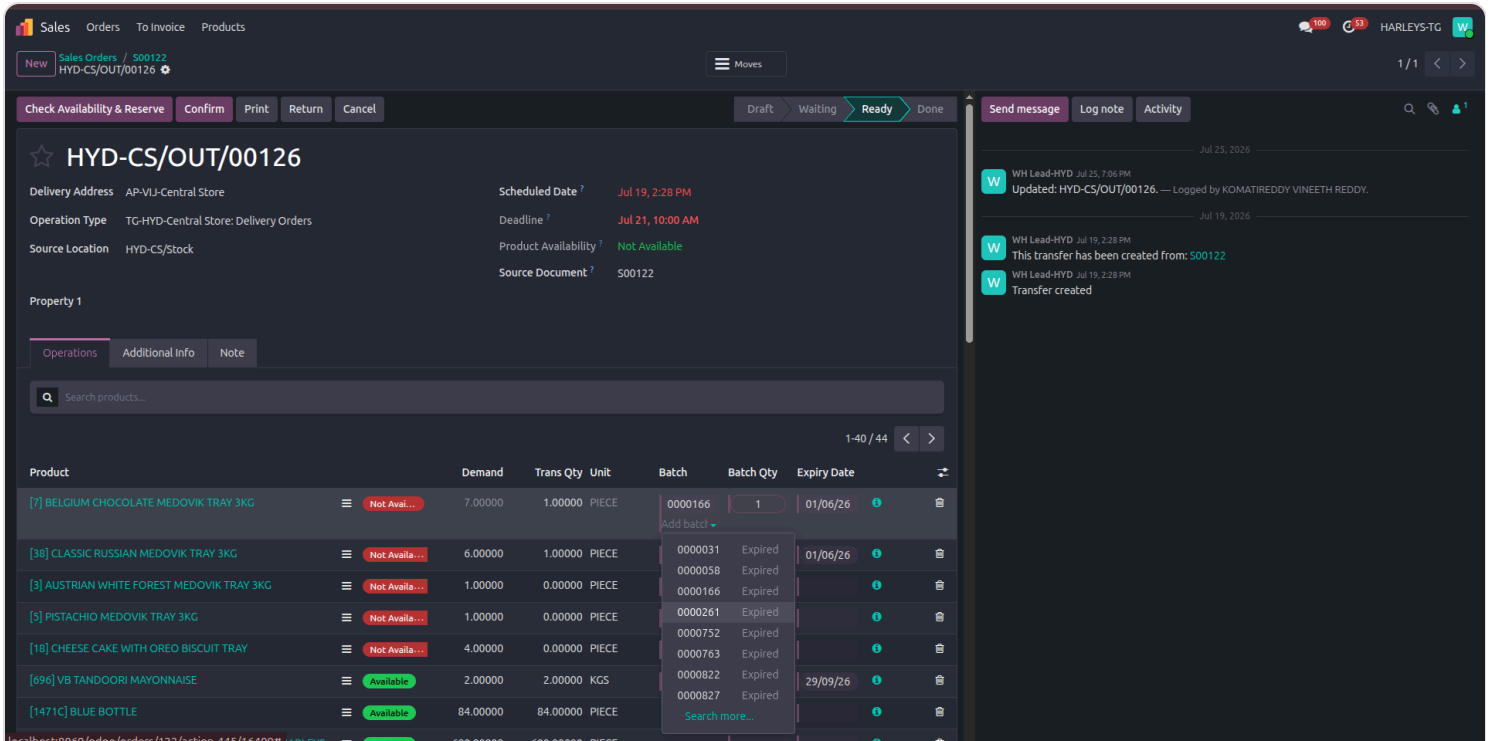
3 Receipt — live search and create share the same popover

The screenshot displays the Odoo Purchase Orders interface. At the top, there's a navigation bar with tabs for Purchase, Orders, Products, Reporting, and Configuration. Below this, a header section shows the document ID 'HYD-CS/IN/00526' and various action buttons like Confirm, Quality Checks, Print, Return, Create e-Waybill / Challan, and Cancel. A right-hand sidebar contains a log of activities, including updates and creations of the document, each logged by 'KOMATIREDDY VINEETH REDDY'.

The main content area features a table with columns: Product, Demand, Trans Qty, Unit, Batch, Batch Qty, and Expiry Date. The first row shows '[1712] AMUL CHEESE BLOCK' with a demand of 52.00000, a unit of KGS, and a batch of 1900. A popover is open over the Batch column, displaying a list of existing batches: 30003421, BAF031, BAF1311, BAF1361, and July1359. At the bottom of the popover is a '+ Create' button.

purchase_orders_select_if_batch_exists.png — typing "3" surfaces matching existing batches (30003421, BAF031, BAF1311, BAF1361, July1359) in the same popover as the "+ Create" button, so a receiving clerk can pick an existing batch or mint a new one from one field, without switching screens.

4 Delivery — select-only, same as the internal flows



`sales_order_delivery.png` — `HYD-CS/OUT/00126`, product **BELGIUM CHOCOLATE MEDOVIK TRAY 3KG**. Like the two internal-transfer screenshots in §1, the dropdown only lists existing batches — no "+ Create" here, confirming Delivery is select-only exactly like Indent/Inventory. Note that every batch listed for this product is marked "Expired" and all remain selectable from this dropdown; nothing in the current view filters expired batches out.

4. Universality — a batch created once is visible everywhere

`stock.lot` is master data, not scoped to the picking that created it. The moment a batch is created on a GRN receipt, it's a normal `stock.lot` row (scoped only by `product_id` / `company_id`) and is immediately selectable from the same product-filtered dropdown on any other picking — internal transfer or delivery — for that product, company-wide. No extra work was needed for this; it falls directly out of `stock.lot` already being one shared model.

NO WAREHOUSE/LOCATION SCOPING ON THE LOT RECORD ITSELF

Availability *by location* is a separate concern, tracked on `stock.quant`, not gated on the lot. A batch created at one warehouse is, today, selectable from the Batch dropdown at any other warehouse for the same product — see §8.

5. Line-item batch add/delete — currently unrestricted

On all four flows, any user who can see the picking and has Odoo's own `stock.group_production_lot` (the base "Lots & Serial Numbers" feature toggle — a feature switch, not a rank/hierarchy group) can freely add or delete Batch rows on any line item (`deleteLot()` / `deleteLine()` in `stock_move_line_batch_field.js`). The only other gate is the standard `is_locked` / `state` check already on the whole Operations tab — batch add/delete disappears once a picking is done and locked, same as the rest of the grid. There is no separate, tighter permission for this action.

6. Not yet built: "only a higher official can change batch details"

STATED REQUIREMENT, NOT CURRENT BEHAVIOR

Flagging this clearly so it isn't documented as already-shipped. Checked directly against the code (`security/ir.model.access.csv` , `security/security_group.xml`): there is no ACL or `ir.rule` on `stock.lot` in this module at all.

Concretely, today:

- Editing an *existing* batch's own fields (name, expiration date, etc.) — via core's "Details" popup (`action_show_details` on `stock.move` , core Odoo) or the standalone Inventory → Products → Lots/Serial Numbers list — is open to anyone who can see the picking or that menu. No extra permission check exists beyond ordinary `stock.lot` access.
- There's already a precedent pattern in this same module for exactly this kind of split, just not wired to batches: `group_indent_admin` / `group_indent_user` (`res.groups.privilege` "Indent Request", `security/security_group.xml`), paired with `ir.rule` domain restrictions (e.g. `indent_request_rule` scoping non-admins to `requested_by = user.id` , `indent_request_manager_rule` giving the admin group unrestricted access). That's the established "user can touch their own records, admin/manager can touch everything" template in this codebase — the natural one to extend for batch-detail editing, as an alternative to reusing core's existing `stock.group_stock_manager` if "higher official" is meant to map to Inventory Manager specifically rather than a new bespoke group.

Before this can be built, needs a decision on: which group/role counts as the "higher official" (existing `stock.group_stock_manager` ? a new group? something per picking type / department?), and exactly what "change batch details" covers — editing the `stock.lot` record's own fields after creation, versus something else

like reassigning which lot a saved move line points to. Line-item add/select/delete (§5) is explicitly *not* meant to be restricted by this — only editing a batch's own details after it exists.

7. Module reference — harleys_customization

FILE	PURPOSE
views/stock_picking_views.xml	Adds the three columns to stock.view_picking_form's Operations tab; all picking_type_code-gated.
static/src/js/stock_move_line_rows.js	Shared row list (getMoveLineRows) all three columns read from.
static/src/js/stock_move_line_batch_field.js / .xml	Batch column: tags, delete, dispatches to inline dropdown (internal/outgoing) or popover (incoming).
static/src/js/stock_move_line_batch_quickcreate.js / .xml	Receipt-only "Add Batch" popover: live search + new-batch + expiry entry.
static/src/js/stock_move_line_row_field.js / .xml	Shared, parametrized widget powering both Batch Qty and Expiry Date columns.
static/src/js/searchable_x2many.js	Operations grid list renderer; also where the new-line auto-save (§8) lives.
static/src/scss/stock_picking_operations.scss	All styling for the above.

No new Python model was added for this feature — it's view + JS/SCSS only, layered on top of Odoo's core stock.picking / stock.move / stock.lot.

8. Known trade-offs / open items

- **Demand quantity (Initial Demand) goes read-only shortly after a new line is added.** Side effect of auto-saving the picking as soon as a product is selected (needed so Batch/Batch Qty/Expiry become usable on that row at all) combined with core Odoo's own _autoconfirm_picking() / is_initial_demand_editable logic, which locks Demand once a move leaves draft state on an is_locked picking. Root-caused but not

yet resolved; the options on the table are (a) scope an override to just-added (`additional = True`) lines specifically, (b) move the auto-save trigger back to "leaving the row" instead of "product selected," or (c) unlock the whole picking (not recommended — that also reopens Scheduled Date editing picking-wide, wider than intended).

- **No warehouse/location scoping on batches (§4)** — a batch created on one GRN is selectable everywhere, company-wide, today. Unbuilt if the business actually wants batches scoped per warehouse/store.
- **"Higher official" governance (§6) is unbuilt** — line-item add/select/delete works for anyone today; editing an existing batch's own details has no extra gate yet.

Internal implementation reference — Harleys Odoo 19 Enterprise. Click any screenshot to view it enlarged and step through the rest. Looking for a non-technical walkthrough instead? See the [User Guide](#).